



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №2
по дисциплине «Технологии и инструментарий машинного обучения»

Студент группы *ИНБО-20-23. Боргачев Т.М.*

(подпись)

Преподаватель *Трушин СМ*

(подпись)

Москва 2025 г.

СОДЕРЖАНИЕ

1 ПРАКТИЧЕСКАЯ РАБОТА №1	3
1.1 Задание 1. Знакомство с данными и EDA	3
1.1.1 Общая характеристика и визуальный обзор	3
1.1.2 Дубликаты и пропуски	15
1.1.3 Работа с числовыми и категориальными признаками	17
1.1.4 Анализ связи признаков с целевой переменной	23
1.1.5 Работа с дисбалансом классов	24
1.1.6 Визуализация структуры данных в пространстве	25
ВЫВОД.....	27

1 ПРАКТИЧЕСКАЯ РАБОТА №1

1.1 Задание 1. Знакомство с данными и EDA

1.1.1 Общая характеристика и визуальный обзор

В качестве данных был выбран датасет Spotify.csv с метаданными о треках с платформы

Ссылка на Google colab с выполненной работой:
<https://colab.research.google.com/drive/11tWiTrygueYnnw-MXVc1zFMF15l415fB?usp=sharing>.

Выполненное задание на Python 1 представлено в Листинге 1.1.

Листинг 1.1 – Задание 1 в Python

```
#Работа будет происходить с датасетом Spotify_modified.csv,
который
#содержит метаданные различных треков с платформы
#искусственно были добавлены выбросы, дубликаты и пропуски
import numpy as np
import pandas as pd
data = pd.read_csv("data/Spotify_modified.csv")
data.head()
# import random
# # Загрузка данных
# data = pd.read_csv("data/Spotify.csv")
# # 1. Добавление дубликатов (5% от общего количества строк)
# n_duplicates = int(len(data) * 0.05)
# duplicate_indices = np.random.choice(data.index, n_duplicates,
replace=False)
# duplicates = data.loc[duplicate_indices].copy()
# data = pd.concat([data, duplicates], ignore_index=True)

# # 2. Добавление пропусков (3% от общего количества ячеек)
# n_missing = int(data.size * 0.03)
# # Определим колонки, в которые можно добавлять пропуски
# missing_columns = ['artists', 'album_name', 'track_name',
'popularity', 'danceability',
# 'energy', 'key', 'loudness', 'mode',
'speechiness', 'acousticness',
```

Продолжение Листинга 1.1

```
#                                'instrumentalness', 'liveness', 'valence',
'tempo', 'track_genre']

# # Генерируем случайные индексы для пропусков
# missing_indices = np.random.choice(data.index, n_missing,
replace=True)
# missing_columns_sample = np.random.choice(missing_columns,
n_missing, replace=True)

# # Добавляем пропуски
# for i in range(n_missing):
#     data.loc[missing_indices[i], missing_columns_sample[i]] =
np.nan

# # 3. Добавление выбросов в числовые колонки
# numeric_columns = ['popularity', 'duration_ms', 'danceability',
'energy', 'key',
#                                'loudness', 'mode', 'speechiness',
'acousticness', 'instrumentalness',
#                                'liveness', 'valence', 'tempo']

# # Добавим выбросы в 2% от общего количества строк
# n_outliers = int(len(data) * 0.02)
# outlier_indices = np.random.choice(data.index, n_outliers,
replace=False)

# for idx in outlier_indices:
#     # Выбираем случайную числовую колонку
#     column = np.random.choice(numeric_columns)

#     # Для разных колонок разные типы выбросов
#     if column in ['danceability', 'energy', 'acousticness',
'liveness', 'valence']:
#         # Эти значения обычно от 0 до 1, сделаем выбросы > 1 или
< 0
#         data.loc[idx, column] = np.random.choice([-0.5, 1.5,
2.0])
#     elif column == 'popularity':
#         # Популярность от 0 до 100
#         data.loc[idx, column] = np.random.choice([-50, 150, 200])
#     elif column == 'duration_ms':
#         # Длительность в миллисекундах, добавим очень
большие/маленькие значения
#         data.loc[idx, column] = np.random.choice([1000, 10000000,
20000000])
#     elif column == 'loudness':
#         # Громкость в дБ, обычно от -60 до 0
#         data.loc[idx, column] = np.random.choice([10, 20, -100])
#     elif column == 'tempo':
#         # Темп в ударах в минуту
#         data.loc[idx, column] = np.random.choice([0, 500, 1000])
```

Продолжение Листинга 1.1

```
# # Проверка результатов
# print("Исходный размер датасета:", len(data) - len(duplicates))
# print("Размер датасета после добавления дубликатов:", len(data))
# print("Количество пропущенных значений:")
# print(data.isnull().sum()[data.isnull().sum() > 0])
# print("\nПример строк с выбросами:")
# print(data.describe())

# # Сохранение модифицированного датасета
# data.to_csv("data/Spotify_modified.csv", index=False)
## 2. Общая характеристика и визуальный обзор
# проверьте типы данных, уникальные значения
print("Типы данных:")
print(data.dtypes)
print("\nРазмер датасета:", data.shape)
# вычислите базовые характеристики для числовых признаков
(медиана, минимум, максимум, стандартное отклонение)
numeric_columns = data.select_dtypes(include=[np.number]).columns
print("Базовые характеристики числовых признаков:")
print(data[numeric_columns].describe())
# для категориальных признаков (частоты значений)
categorical_columns =
data.select_dtypes(include=['object']).columns
print("\nКатегориальные признаки и их частоты:")
for col in categorical_columns:
    print(f"\n{col}:")
    print(data[col].value_counts().head(10))
# постройте распределения числовых признаков
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(4, 4, figsize=(16, 12))
axes = axes.ravel()

for i, col in enumerate(numeric_columns[1:]):
    if i < len(axes):
        sns.histplot(data=data, x=col, bins=30, kde=True,
ax=axes[i], color='skyblue')
        axes[i].set_title(f'{col}')

plt.tight_layout()
plt.show()
print(categorical_columns)
# постройте диаграммы частот категориальных признаков
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.ravel()

top_genres = data['track_genre'].value_counts().head(10)
axes[0].bar(range(len(top_genres)), top_genres.values,
color='lightcoral')
axes[0].set_title('Топ 10 жанров')
```

Продолжение Листинга 1.1

```
axes[0].set_xticks(range(len(top_genres)))
axes[0].set_xticklabels(top_genres.index, rotation=45)

top_artists = data['artists'].value_counts().head(20)
axes[1].bar(range(len(top_artists)), top_artists.values,
color='lightgreen')
axes[1].set_title('Топ 20 артистов')
axes[1].set_xticks(range(len(top_artists)))
axes[1].set_xticklabels(top_artists.index, rotation=45)

top_artists = data['album_name'].value_counts().head(15)
axes[2].bar(range(len(top_artists)), top_artists.values,
color='magenta')
axes[2].set_title('Топ 15 альбомов')
axes[2].set_xticks(range(len(top_artists)))
axes[2].set_xticklabels(top_artists.index, rotation=45)

top_artists = data['track_name'].value_counts().head(10)
axes[3].bar(range(len(top_artists)), top_artists.values,
color='red')
axes[3].set_title('Топ 10 треков')
axes[3].set_xticks(range(len(top_artists)))
axes[3].set_xticklabels(top_artists.index, rotation=45)
plt.tight_layout()
plt.show()
# выведите количество уникальных значений по каждому признаку
print("Количество уникальных значений по признакам:")
unique_counts = data.nunique().sort_values(ascending=False)
print(unique_counts)
# подсчитайте долю пропусков по каждому признаку и отсортируйте
признаки по убыванию доли
missing_percent = (data.isnull().sum() / len(data)) * 100
missing_percent = missing_percent[missing_percent >
0].sort_values(ascending=False)
print("Доля пропусков по признакам (%):")
print(missing_percent)
```

Первые строки датасета представлены на Рисунке 1.

index	track_id	artists	album_name	track_name	popularity	duration_ms	explicit	danceability	energy	...	loudness	mode	speechiness	acousticness	instrumental
0	5SuOikwiRyPMVoIQDJUgSV	Gen Hoshino	Comedy	Comedy	73.0	230666	False	0.676	0.4610	...	-6.746	0.0	0.1430	0.0322	0.00
1	4qPND8W13p13qLC10KI3A	Ben Woodward	Ghost (Acoustic)	Ghost - Acoustic	55.0	149610	False	0.420	0.1660	...	NaN	1.0	0.0763	0.9240	0.00
2	1tJBSr7s7jYXzM8EGcbK5b	Ingrid Michaelson; ZAYN	To Begin Again	To Begin Again	57.0	210826	False	0.438	0.3590	...	-9.734	1.0	0.0557	0.2100	0.00
3	6lfxq3CG4xtIEg7opyCyx	Kina Grannis	Crazy Rich Asians (Original Motion Picture Sou...	Can't Help Falling In Love	71.0	201933	False	0.266	0.0596	...	-18.515	1.0	0.0363	0.9050	0.00
4	5vjLSffimilP26QG5WcN2K	Chord Overstreet	Hold On	Hold On	82.0	198853	False	0.618	0.4430	...	-9.681	1.0	0.0526	0.4690	0.00

Рисунок 1 – Датасет Spotify_modified.csv

Информация о типах данных, полученная с помощью Python, представлена на Рисунке 2.

```
Типы данных:
index          int64
track_id       object
artists        object
album_name     object
track_name     object
popularity     float64
duration_ms    int64
explicit       bool
danceability   float64
energy         float64
key            float64
loudness       float64
mode           float64
speechiness    float64
acousticness   float64
instrumentalness float64
liveness       float64
valence        float64
tempo          float64
time_signature int64
track_genre    object
dtype: object

Размер датасета: (119700, 21)
```

Рисунок 2 – Типы данных Python

Проверка типов данных и уникальных значений в Logiном представлена на Рисунке 3.

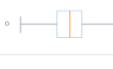
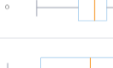
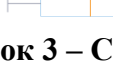
Статистика										
Уникальные значения Порядковые статистики										
№	Метка	Вид	Гистограмма	Диаграмма размаха	Минимум	Максимум	Среднее	Стандарт...	Пропуски	Уникальн...
1	ab track_id		Число значений - ...	Недоступно	22	22	22	0	0	89741
2	ab artists		Число значений - ...	Недоступно	0	513	15,668529...	13,400750...	0	30774
3	ab album_name		Число значений - ...	Недоступно	0	243	19,373833...	13,990527...	21	45566
4	ab track_name		Число значений - ...	Недоступно	0	511	17,290367...	13,355596...	0	71539
5	90 popularity				-50	200	33,334360...	22,830670...	4552	
6	12 duration_ms				0	20000000	245199	549765,31...	0	
7	01 explicit			Недоступно	Ложь	Истина	Истина	0,2802023...	0	2
8	90 danceability				-0,5	2	0,5674752...	0,1794264...	4661	
9	90 energy				-0,5	2	0,6416234...	0,2551985...	4661	
10	90 key				0	11	5,3067675...	3,5621217...	4740	

Рисунок 3 – Статистика данных Loginom

Описание числовых характеристик с помощью Python представлено на Рисунке 4 и Рисунке 5.

Базовые характеристики числовых признаков:

	index	popularity	duration_ms	danceability	\
count	119700.000000	115148.000000	1.197000e+05	115039.000000	
mean	57005.637636	33.334361	2.451991e+05	0.567475	
std	32913.784312	22.830671	5.497653e+05	0.179426	
min	0.000000	-50.000000	0.000000e+00	-0.500000	
25%	28505.750000	17.000000	1.741010e+05	0.456000	
50%	56988.500000	35.000000	2.130000e+05	0.581000	
75%	85510.250000	50.000000	2.617860e+05	0.695000	
max	113999.000000	200.000000	2.000000e+07	2.000000	

	energy	key	loudness	mode	\
count	115039.000000	114960.000000	115089.000000	115139.000000	
mean	0.641623	5.306768	-8.277269	0.637091	
std	0.255199	3.562122	5.417862	0.480841	
min	-0.500000	0.000000	-100.000000	0.000000	
25%	0.471000	2.000000	-10.019000	0.000000	
50%	0.684000	5.000000	-7.005000	1.000000	
75%	0.854000	8.000000	-4.997000	1.000000	
max	2.000000	11.000000	20.000000	1.000000	

	speechiness	acousticness	instrumentalness	liveness	\
count	115156.000000	115072.000000	115002.000000	115073.000000	
mean	0.084536	0.316157	0.155984	0.215012	
std	0.105568	0.336002	0.309618	0.198133	
min	0.000000	-0.500000	0.000000	-0.500000	
25%	0.035900	0.016900	0.000000	0.098000	
50%	0.048900	0.169000	0.000041	0.132000	
75%	0.084400	0.599000	0.048600	0.274000	
max	0.965000	2.000000	1.000000	2.000000	

Рисунок 4 – Описание признаков Python. Часть 1

	valence	tempo	time_signature
count	115195.000000	115080.000000	119700.000000
mean	0.475019	122.858521	3.904027
std	0.263134	38.418683	0.432394
min	-0.500000	0.000000	0.000000
25%	0.260000	99.197000	4.000000
50%	0.464000	122.023000	4.000000
75%	0.684000	140.091000	4.000000
max	2.000000	1000.000000	5.000000

Рисунок 5 – Описание признаков Python. Часть 2

Гистограмма распределения количества слов в песнях представлена на Рисунке 6.

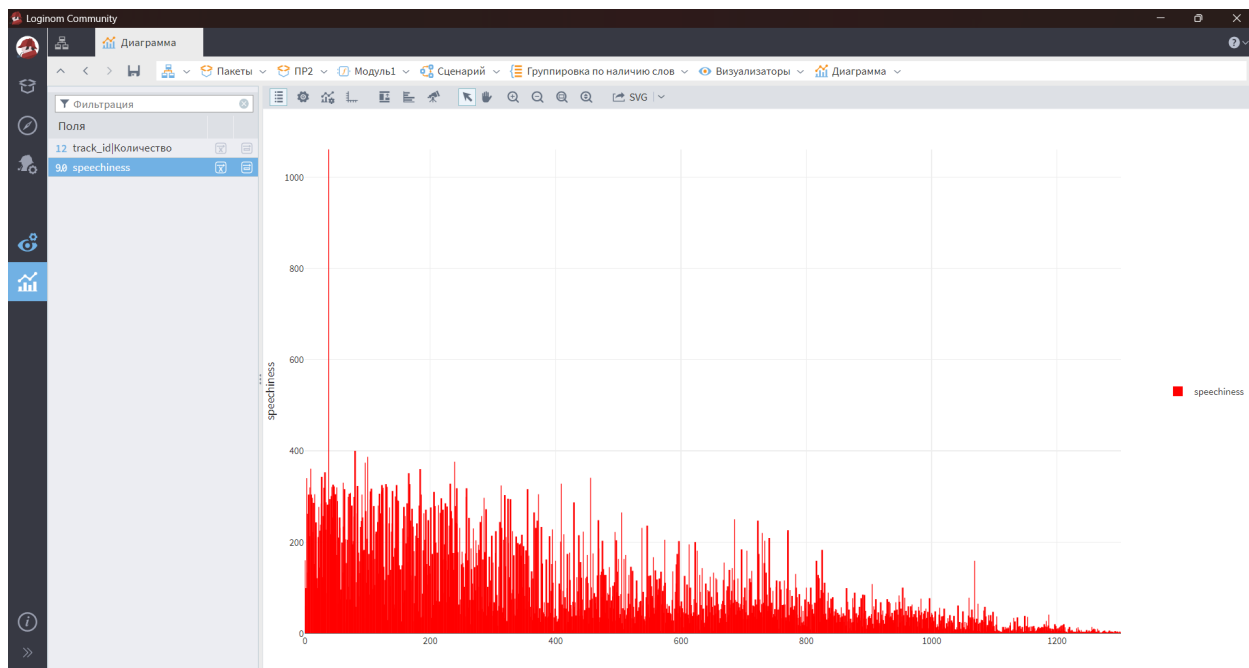


Рисунок 6 – Гистограмма распределения speechiness

Гистограмма распределения популярности песен представлена на Рисунке 7.

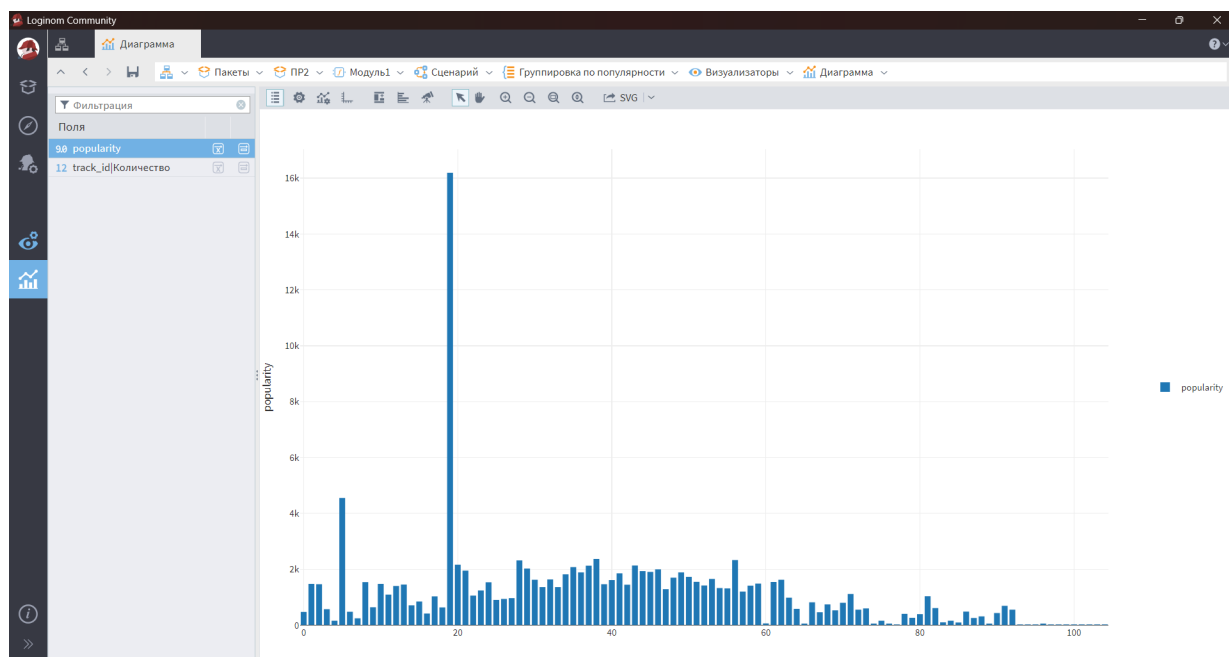


Рисунок 7 – Гистограмма распределения popularity

Гистограмма распределения длительности песен представлена на Рисунке 8.

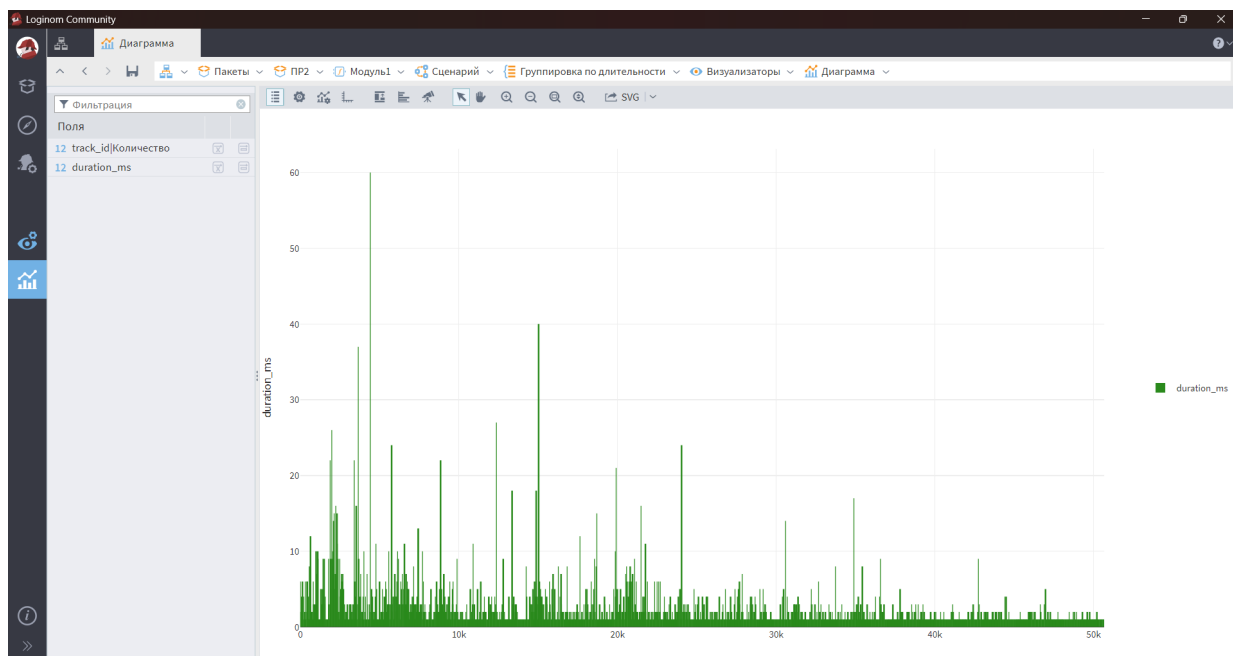


Рисунок 8 – Гистограмма распределения duration

Диаграммы распределения числовых признаков, построенные в Python, представлены на Рисунке 9.

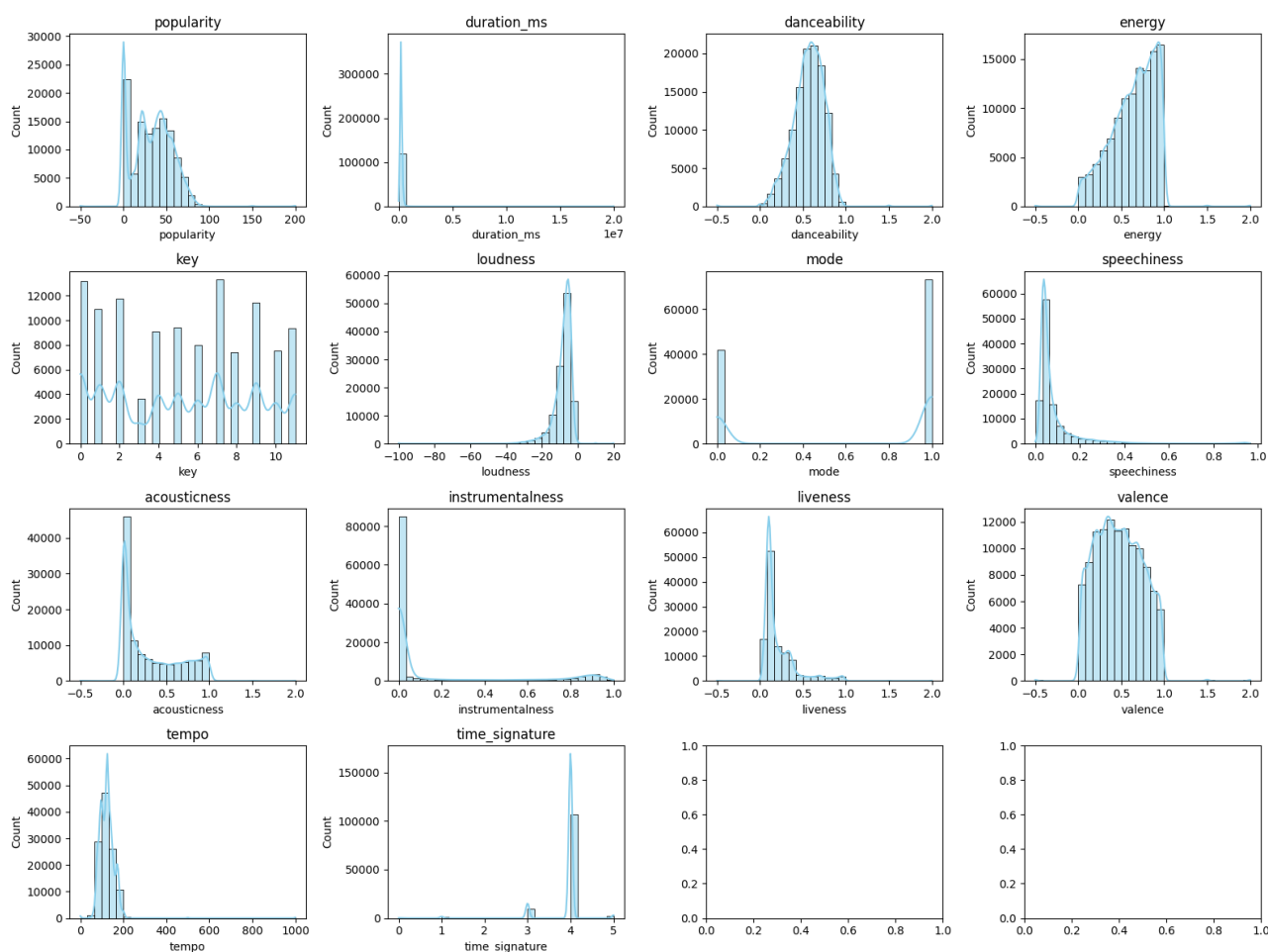


Рисунок 9 – Гистограммы распределения Python

Столбчатые диаграммы распределения для категориальных данных, построенные в Python, представлены на Рисунке 10.

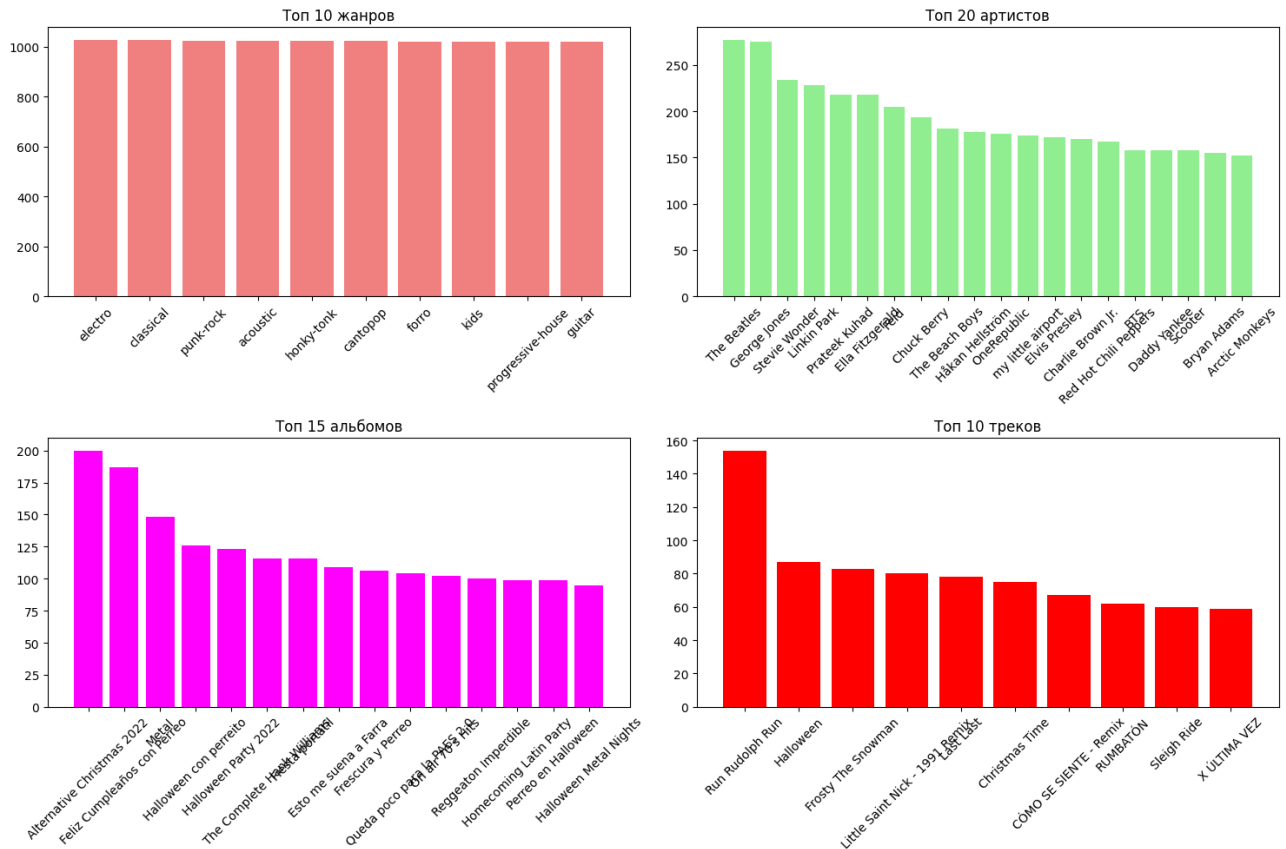


Рисунок 10 – Диаграммы категориальных признаков Python

Количество уникальных значений, посчитанных в Logisim, представлено на Рисунке 11.

The screenshot shows the Logisim software interface with a table titled "Свертка столбцов • Быстрый просмотр" (Column Pivot • Quick View). The table displays the number of unique values for 19 different features. The first column is labeled "#", the second column is labeled "ab Поля" (ab Fields), and the third column is labeled "12 Уникальные значения" (12 Unique values).

#	ab Поля	12 Уникальные значения
1	popularity Кол-во уникальных	104
2	duration_ms Кол-во уникальных	50 654
3	explicit Кол-во уникальных	2
4	danceability Кол-во уникальных	1 170
5	energy Кол-во уникальных	2 067
6	key Кол-во уникальных	12
7	loudness Кол-во уникальных	19 269
8	mode Кол-во уникальных	2
9	speechiness Кол-во уникальных	1 485
10	acousticness Кол-во уникальных	5 048
11	instrumentalness Кол-во уникальных	5 338
12	liveness Кол-во уникальных	1 723
13	valence Кол-во уникальных	1 787
14	tempo Кол-во уникальных	44 720
15	time_signature Кол-во уникальных	5
16	artists Кол-во уникальных	30 774
17	album_name Кол-во уникальных	45 566
18	track_name Кол-во уникальных	71 541
19	track_genre Кол-во уникальных	115

Рисунок 11 – Количество уникальных значений

Количество уникальных значений, посчитанное в Python представлено на Рисунке 12.

```
Количество уникальных значений по признакам:
index            114000
track_id         89741
track_name       71540
duration_ms      50654
album_name       45566
tempo            44720
artists          30773
loudness         19269
instrumentalness  5338
acousticness     5048
energy           2067
valence          1787
liveness         1723
speechiness      1485
danceability     1170
track_genre      114
popularity       104
key              12
time_signature   5
mode             2
explicit         2
dtype: int64
```

Рисунок 12 – Уникальные значения Python

Доля пропусков в Logiном представлена на Рисунке 13.

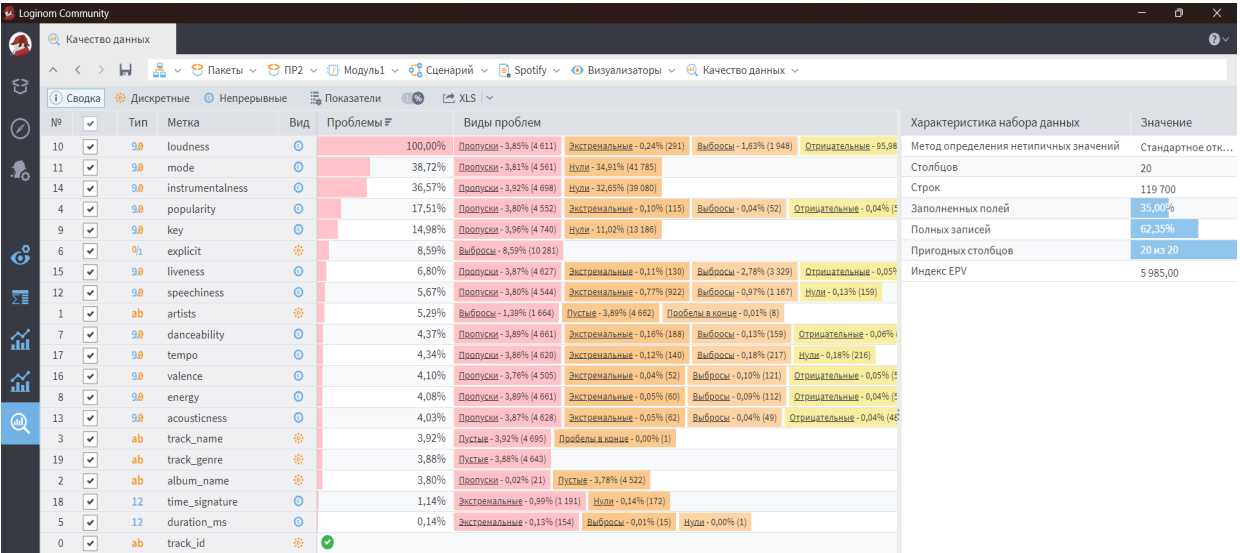


Рисунок 13 – Доля пропусков Logiном

Доля пропусков по признакам в Python, представлена на Рисунке 14.

```
Доля пропусков по признакам (%):  
key                3.959900  
instrumentalness    3.924812  
track_name         3.922306  
artists            3.894737  
energy             3.893901  
danceability       3.893901  
track_genre        3.878864  
acousticness       3.866332  
liveness           3.865497  
tempo              3.859649  
loudness           3.852130  
mode               3.810359  
popularity         3.802840  
speechiness        3.796157  
album_name         3.777778  
valence            3.763576  
dtype: float64
```

Рисунок 14 – Доля пропусков Python

Сравнение Python и Loginom для первичного анализа:

Преимущества Python:

- большая гибкость и контроль над анализом;
- богатая экосистема библиотек (pandas, numpy, matplotlib, seaborn);
- возможность автоматизации и воспроизводимости;
- подходит для сложных аналитических задач;
- расширяемость и интеграция с другими инструментами.

Недостатки Python:

- требует программирования;
- необходимость писать код для каждой визуализации.

Преимущества Loginom:

- интерфейс "drag-and-drop";
- быстрое создание стандартных визуализаций;

- минимальные требования к программированию;
- хорош для бизнес-пользователей;
- встроенные шаблоны анализа.

Недостатки Loginom:

- меньшая гибкость;
- ограниченные возможности для сложных анализов;
- зависимость от графического интерфейса;
- меньше возможностей для автоматизации.

Для быстрого первичного анализа Loginom может быть удобнее, но для глубокого анализа и кастомизации Python предпочтительнее.

1.1.2 Дубликаты и пропуски

Подсчёт дубликатов представлен на Рисунке 15.

```
Количество полных дубликатов: 1611
Количество частичных дубликатов (по track_id): 29959
```

Рисунок 15 – Дубликаты

Результат удаления дубликатов представлен на Рисунке 16.

```
Изначальное количество записей: 119700
Удалено дубликатов: 1611
Финальное количество записей: 118089
```

Рисунок 16 – Удаление дубликатов

Полные дубликаты не несут новой информации, а потому подлежат удалению, агрегировать их не имеет смысла.

Частичные же дубликаты могут быть валидными, если:

1. Один и тот же трек записан в разных альбомах.
2. Разные версии одного трека (live, acoustic и т.д.).
3. Ошибки в данных (один трек с разными ID).
4. Агрегированные данные из разных источников.

Их лучше исследовать отдельно.

Пропуски представлены на Рисунке 17.



```
Признаки с пропущенными значениями:  
artists          4662  
album_name       4522  
track_name       4695  
popularity       4552  
danceability     4661  
energy           4661  
key              4740  
loudness         4611  
mode             4561  
speechiness      4544  
acousticness     4628  
instrumentalness 4698  
liveness         4627  
valence          4505  
tempo           4620  
track_genre      4643  
dtype: int64
```

Рисунок 17 – Признаки с пропущенными значениями

Обработаем пропуски. Для категориальных признаков - выбрано удаление, так как это позволяет сохранить достоверность данных без искусственного заполнения. Для числовых данных выбрана медиана, как самая устойчивая к выбросам, и так как она характеризует "типичное" значение.

Результат обработки пропусков представлен на Рисунке 18.


```
Заполнен artists модой: The Beatles
Заполнен album_name модой: Alternative Christmas 2022
Заполнен track_name модой: Run Rudolph Run
Заполнен track_genre модой: electro
Заполнен popularity медианой: 35.0
Заполнен danceability медианой: 0.581
Заполнен energy медианой: 0.684
Заполнен key медианой: 5.0
Заполнен loudness медианой: -7.005
Заполнен mode медианой: 1.0
Заполнен speechiness медианой: 0.0489
Заполнен acousticness медианой: 0.169
Заполнен instrumentalness медианой: 4.12e-05
Заполнен liveness медианой: 0.132
Заполнен valence медианой: 0.464
Заполнен tempo медианой: 122.023
Все пропуски обработаны
```

Рисунок 18 – Обработка пропусков

Таким образом, дубликаты и пропуски обработаны.

1.1.3 Работа с числовыми и категориальными признаками

Закодируем категориальные признаки. Для номинальных категориальных признаков используем one-hot encoding. Так как artists и track_name имеют много уникальных значений, используем Label Encoding. Для track_genre используем one-hot encoding, так как это ограниченное число категорий.

Результат кодирования представлен на Рисунке 19.

Категориальные признаки закодированы
One-hot encoding применен для track_genre

	artists_encoded	album_name_encoded	track_name_encoded
0	9572	7416	10458
1	3030	13505	20051
2	11440	35715	54171
3	13708	7856	8507
4	4858	15421	22888

Рисунок 19 – Результат кодирования категориальных признаков

Найдем и опишем выбросы, построив диаграммы размаха. Результат постройки представлен на Рисунке 20.

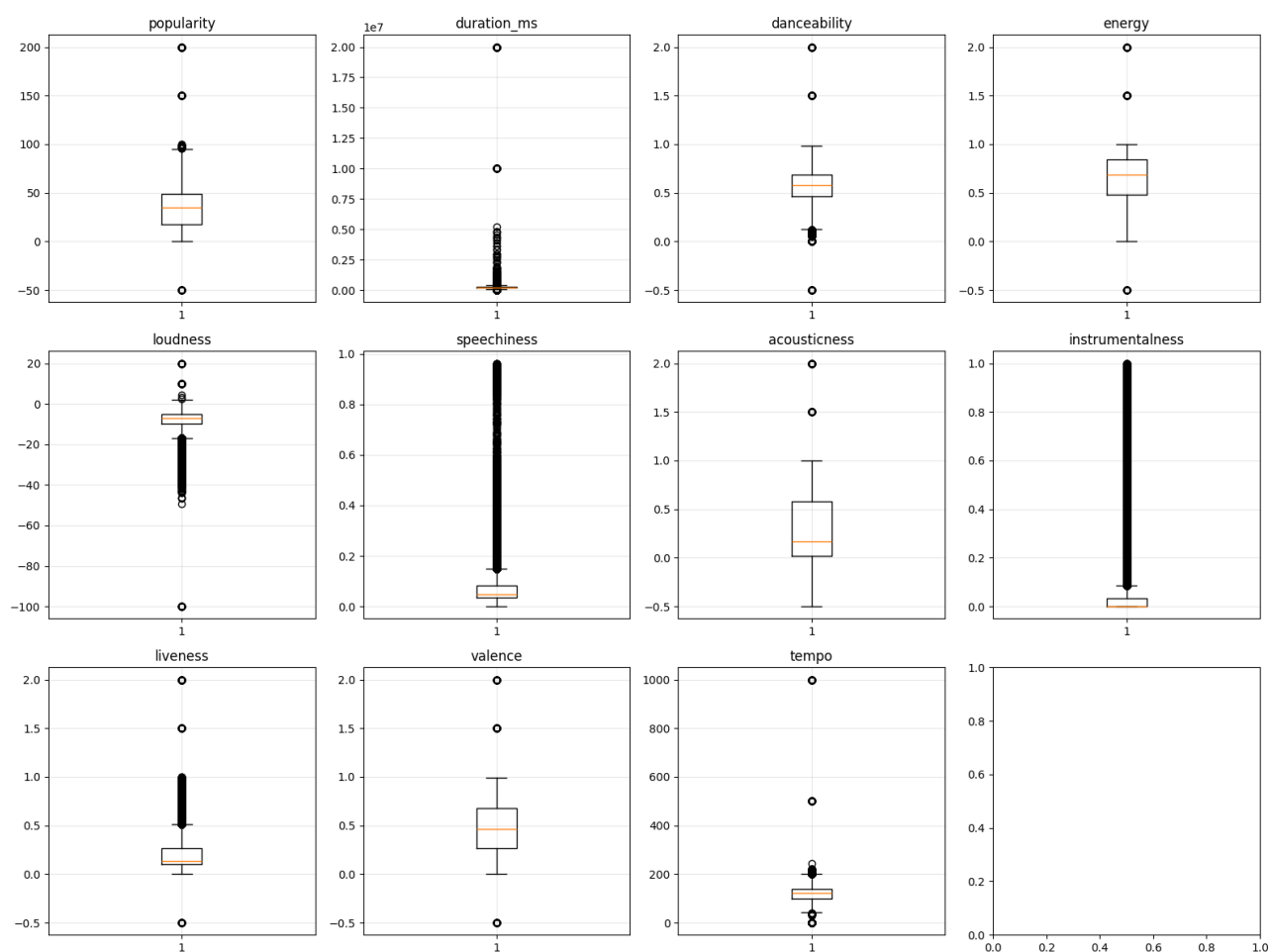


Рисунок 20 – Построенные boxplots

Самое большое число выбросов присутствует у характеристик треков: duration, loudness, speechiness, instrumentality, liveness. Впрочем, огромное количество выбросов speechiness и instrumentality может и не являться таковым, а быть валидными значениями, так как в зависимости от жанра музыки, длительности и желаний авторов, подобные параметры могут очень сильно разниться.

Подсчитанное количество выбросов по правилу 3σ и IQR представлено на Рисунке 21.

```
popularity: выбросов по  $3\sigma$  = 147
duration_ms: выбросов по  $3\sigma$  = 144
danceability: выбросов по  $3\sigma$  = 280
energy: выбросов по  $3\sigma$  = 144
loudness: выбросов по  $3\sigma$  = 2014
speechiness: выбросов по  $3\sigma$  = 1888
acousticness: выбросов по  $3\sigma$  = 99
instrumentality: выбросов по  $3\sigma$  = 0
liveness: выбросов по  $3\sigma$  = 3121
valence: выбросов по  $3\sigma$  = 145
tempo: выбросов по  $3\sigma$  = 289
popularity: выбросов по IQR = 165
duration_ms: выбросов по IQR = 5153
danceability: выбросов по IQR = 937
energy: выбросов по IQR = 144
loudness: выбросов по IQR = 5996
speechiness: выбросов по IQR = 12218
acousticness: выбросов по IQR = 99
instrumentality: выбросов по IQR = 22919
liveness: выбросов по IQR = 8001
valence: выбросов по IQR = 145
tempo: выбросов по IQR = 820
```

Рисунок 21 – Подсчет количества выбросов

Правило трёх сигм допускает валидность instrumentality, в то время как межквартильный размах считает этот признак самым нестабильным.

Не будем удалять выбросы полностью, а используем ограничение: для признаков типа loudness, tempo - ограничение на 1% и 99% перцентили. Для

музыкальных характеристик (danceability, energy и т.д.) – оставить как есть, так как экстремальные значения могут быть валидными (очень энергичная или тихая музыка).

Обоснование: выбросы в музыкальных данных часто представляют собой реальные особенности (очень популярные треки, очень громкие записи и т.д.) и могут быть важны для анализа.

Асимметрия числовых признаков представлена на Рисунке 22.

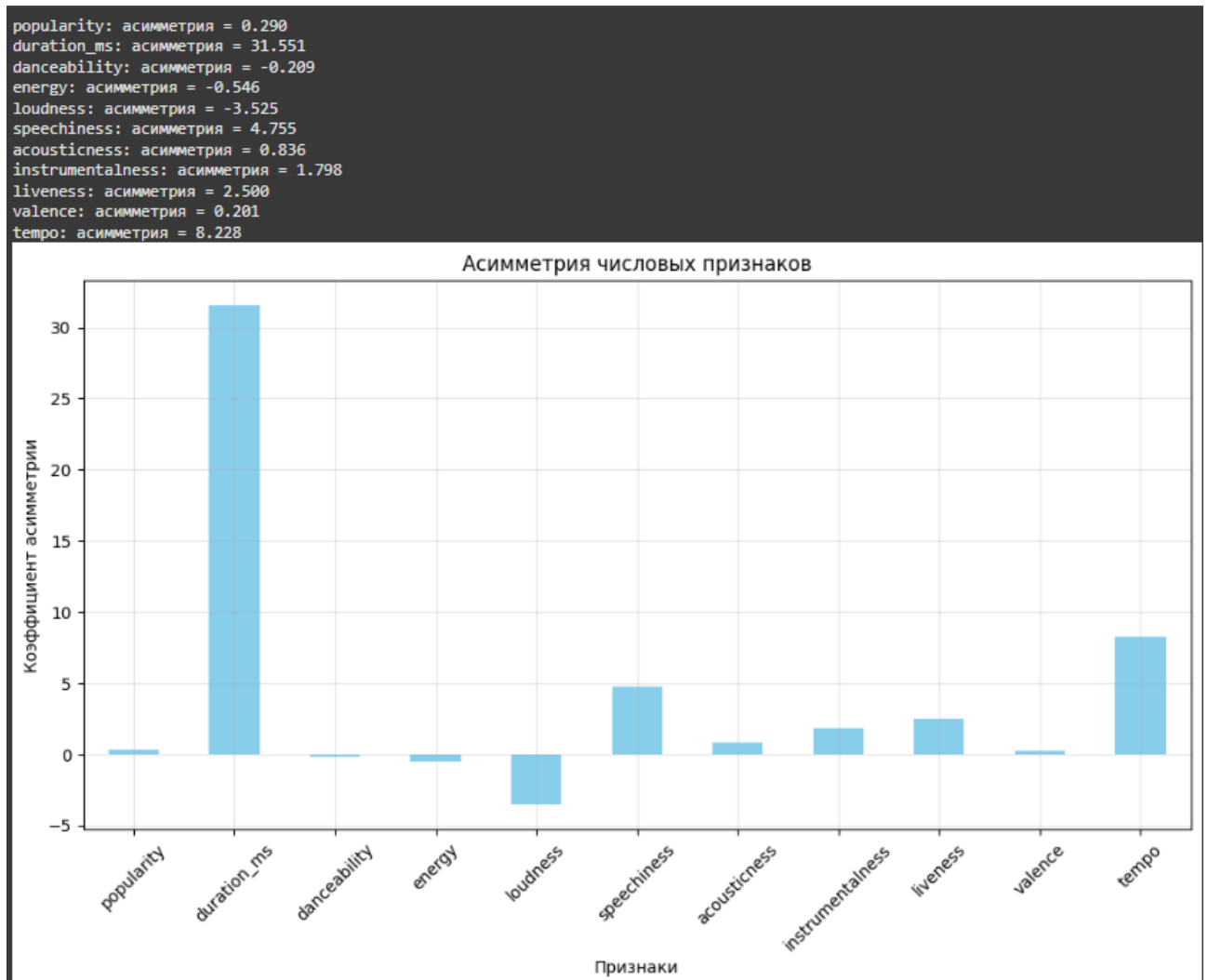


Рисунок 22 – Асимметрия числовых признаков

Выбор масштабирования:

1. Сильно асимметричные признаки ($|\text{skew}| > 1$): используем RobustScaler.
2. Умеренно асимметричные признаки ($0.5 < |\text{skew}| < 1$): применим MinMax.

3. Близкие к нормальным ($|\text{skew}| < 0.5$): danceability, energy, valence, tempo: возьмем StandardScaler.

Обоснование выбора StandardScaler + RobustScaler + MinMaxScaler:

- standardScaler для нормально распределенных признаков;
- robustScaler для признаков с выбросами.

Такой выбор обеспечит лучшую устойчивость к выбросам и сохранит структуру данных.

Результат обработки выбросов признаков с самым большим их количеством представлен на Рисунке 23.

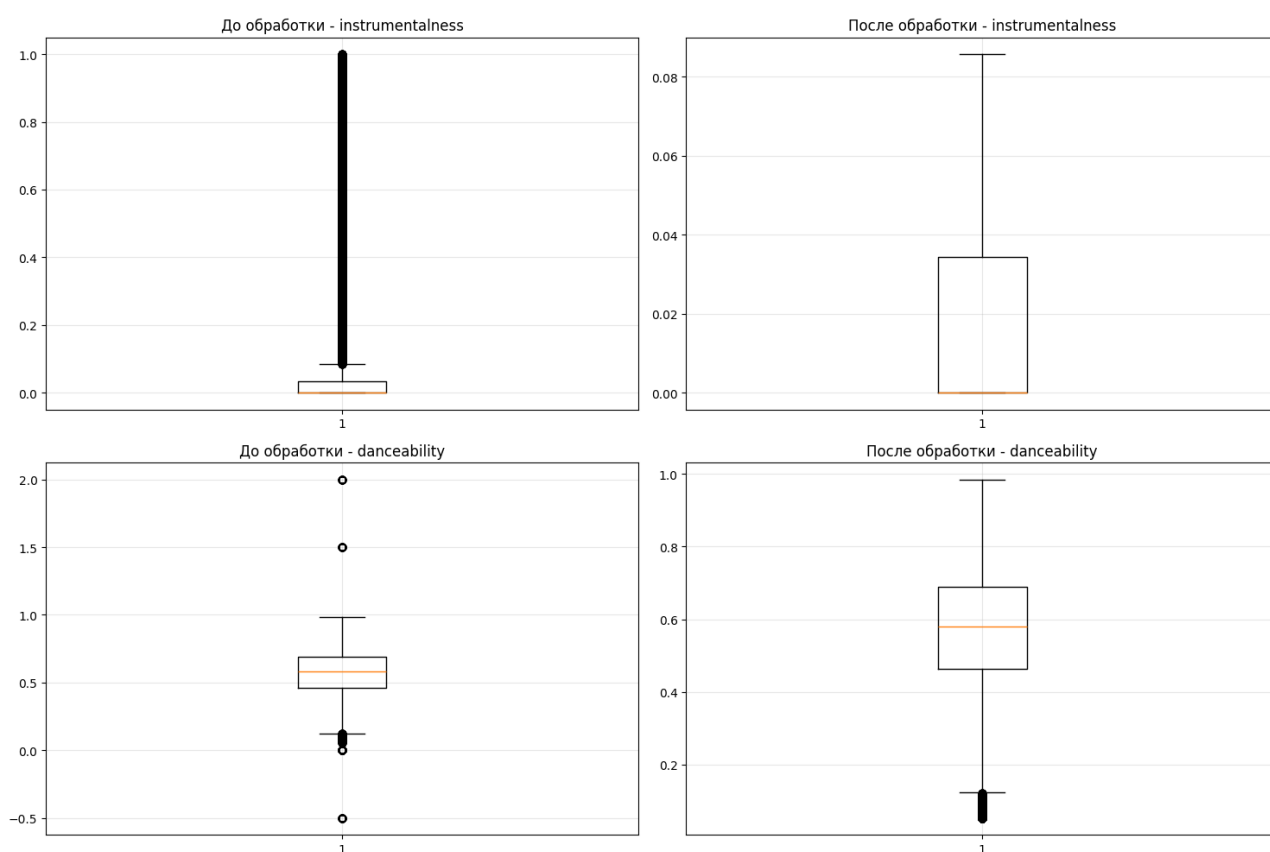


Рисунок 23 — Выбросы признаков до и после обработки

Результат масштабирования признаков представлен на Рисунке 24.

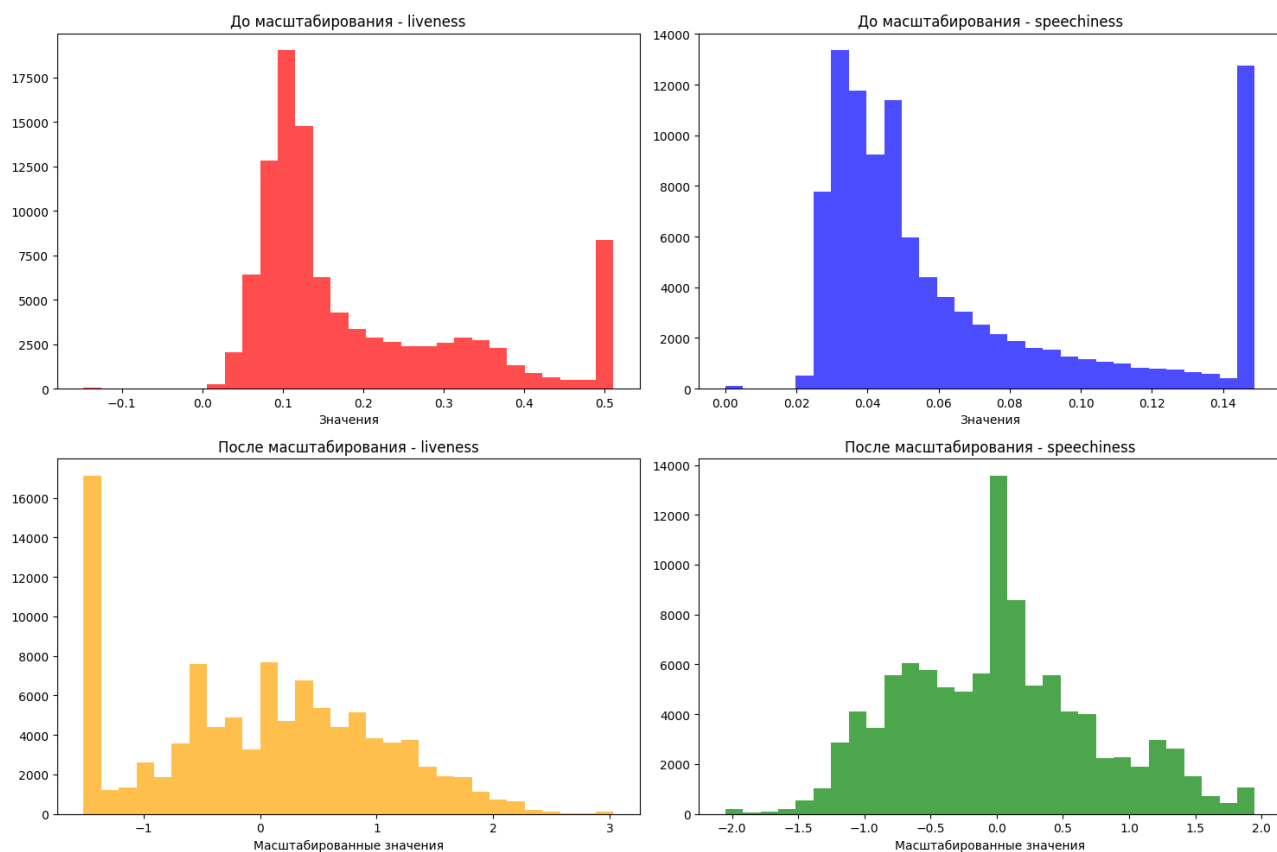


Рисунок 24 — Масштабирование признаков

Распределение признаков после обработки стало больше похоже на нормальное

Сравнение статистик до и после обработки представлено на Рисунке 25.

```

=== ИЗМЕНЕНИЯ ===
popularity: изменение среднего = 0.056244, изменение std = 0.444442
duration_ms: изменение среднего = 22667.704176, изменение std = 478969.795400
danceability: изменение среднего = 0.000320, изменение std = 0.004929
energy: изменение среднего = 0.000413, изменение std = 0.003537
loudness: изменение среднего = 0.355631, изменение std = 1.406024
speechiness: изменение среднего = 0.017533, изменение std = 0.063436
acousticness: изменение среднего = 0.000588, изменение std = 0.002969
instrumentalness: изменение среднего = 0.128246, изменение std = 0.269177
liveness: изменение среднего = 0.019164, изменение std = 0.059998
valence: изменение среднего = 0.000465, изменение std = 0.003108
tempo: изменение среднего = 0.582219, изменение std = 8.085895

```

Рисунок 25 — Изменения среднего и стандартного отклонения признаков

Таким образом, был уменьшен разброс всех обработанных признаков.

1.1.4 Анализ связи признаков с целевой переменной

Построим тепловую карту корреляции числовых признаков. Построенная карта представлена на Рисунке 26.

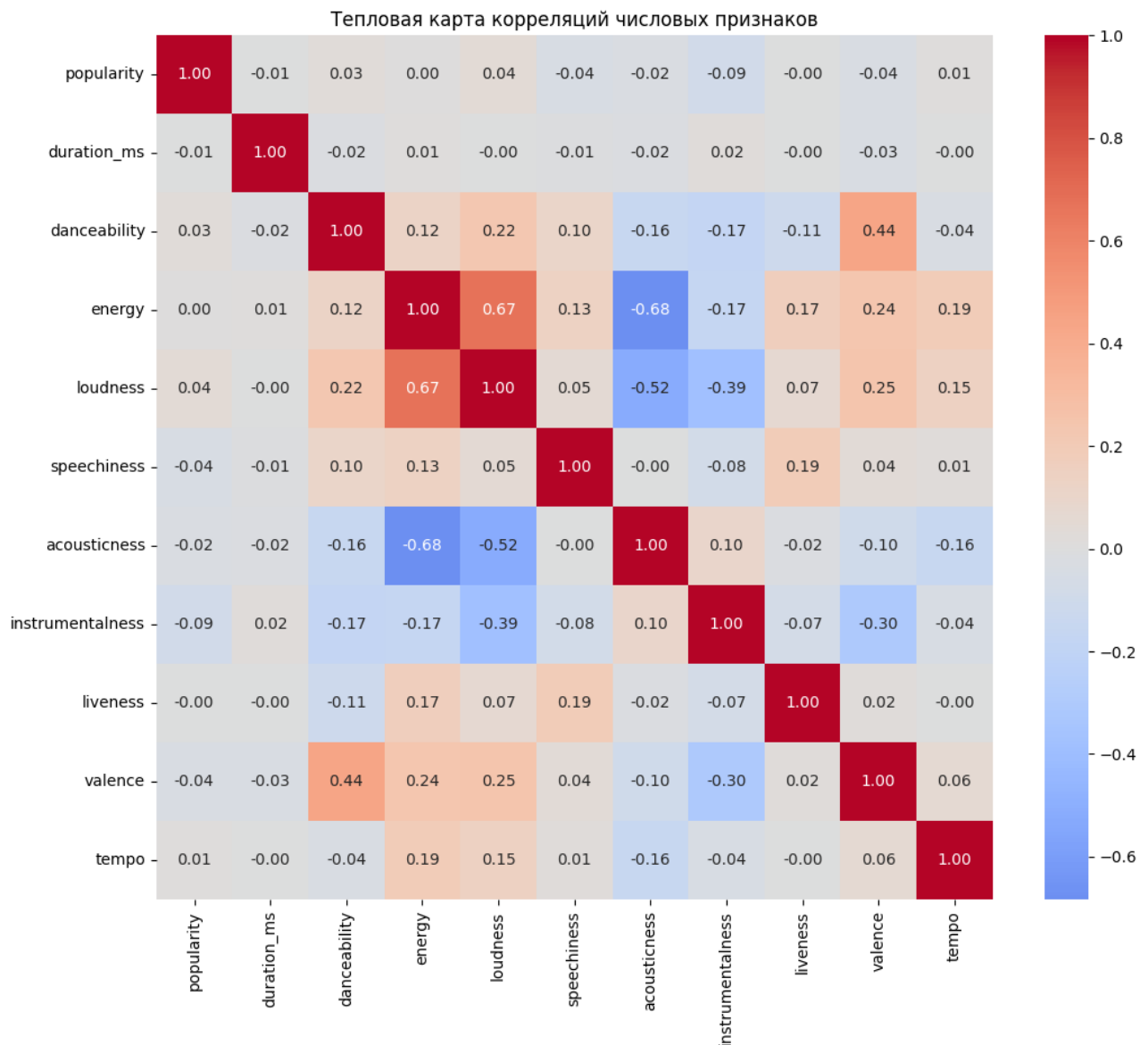


Рисунок 26 — Тепловая карта корреляции

Выберем целевой переменной показатель popularity. Все признаки по отдельности мало коррелируют с популярностью.

Сильно коррелирующие между друг другом признаки: energy, acousticness и loudness.

Построенный scatterplot для самых коррелирующих признаков представлен на Рисунке 27.

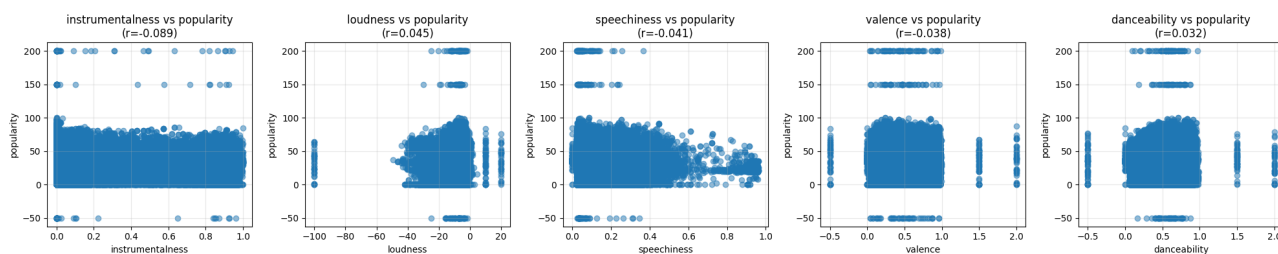


Рисунок 27 — Диаграмма рассеивания для коррелирующих признаков

Результат ANOVA-анализа категориальных признаков с числовым представлен на Рисунке 28.

```
ANOVA анализ для категориальных признаков:
artists_encoded: F-stat=5.628, p-value=0.000000
album_name_encoded: F-stat=12.764, p-value=0.000000
explicit: F-stat=169.479, p-value=0.000000
mode: F-stat=13.020, p-value=0.000308
```

Рисунок 28 — Результат ANOVA-анализа

Категориальные признаки так же слабо связаны с целевой.

1.1.5 Работа с дисбалансом классов

Визуализация проверки на дисбаланс классов представлен на Рисунке 29.

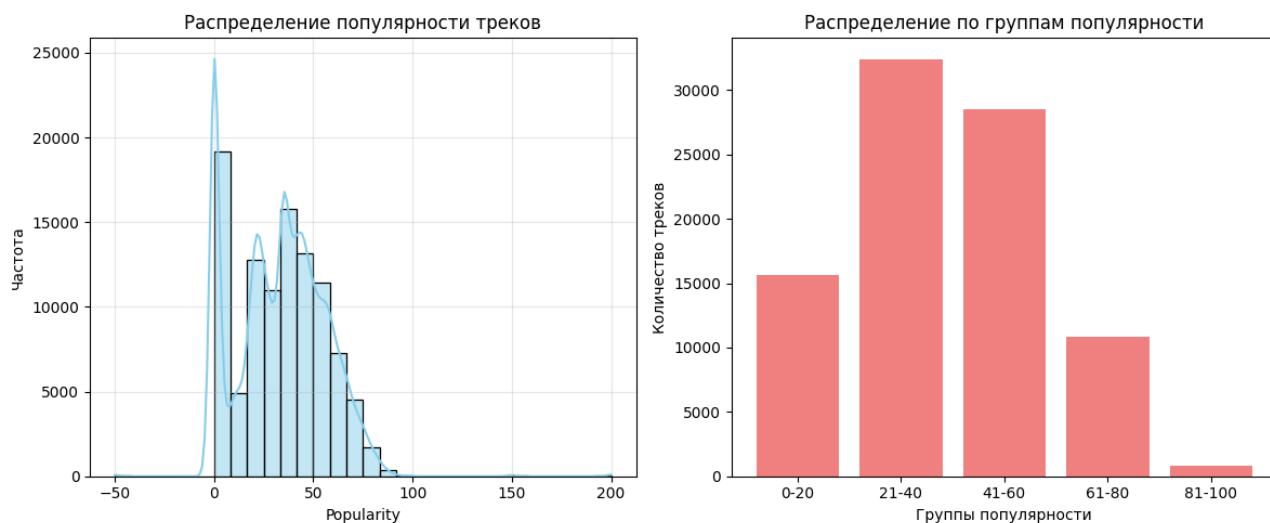


Рисунок 29 – Распределение популярности треков

Популярных треков в несколько раз меньше, чем средней и малой популярности. Соответственно наблюдается дисбаланс классов. Высоко

популярные: 908. Низко популярные: 101325. Соотношение: 1 к 111.6.

Добавим синтетически данные для популярных треков, а также шум. Результат балансировки представлен на Рисунке 30.

```
РЕЗУЛЬТАТЫ БАЛАНСИРОВКИ:  
Исходные данные: 102233 записей  
Сгенерировано синтетических: 49754 записей  
Итоговые данные: 151987 записей  
Высокопопулярные до: 908 (0.9%)  
Высокопопулярные после: 50662 (33.3%)  
Новое соотношение: 1 к 2.0  
Улучшение соотношения: с 1 к 111.6 до 1 к 2.0
```

Рисунок 30 — Результат балансировки классов

Визуализация классов до и после балансировки представлена на Рисунке 31.

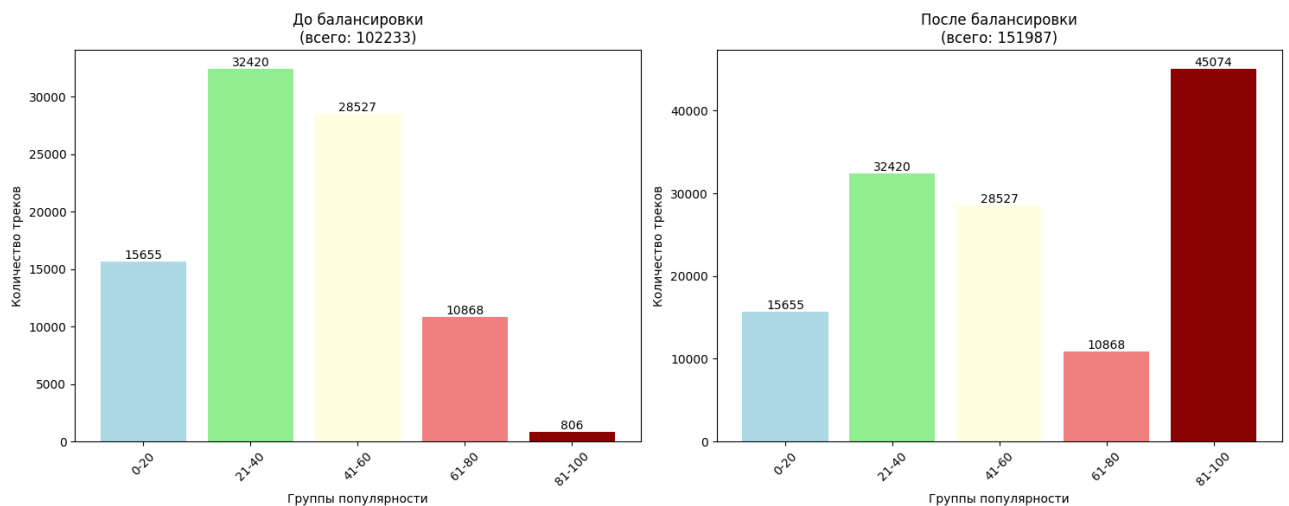


Рисунок 31 — Визуализация балансировки

Таким образом, улучшилось соотношение популярных треков к непопулярным.

1.1.6 Визуализация структуры данных в пространстве

PCA и t-SNE проекции представлены на Рисунке 32.

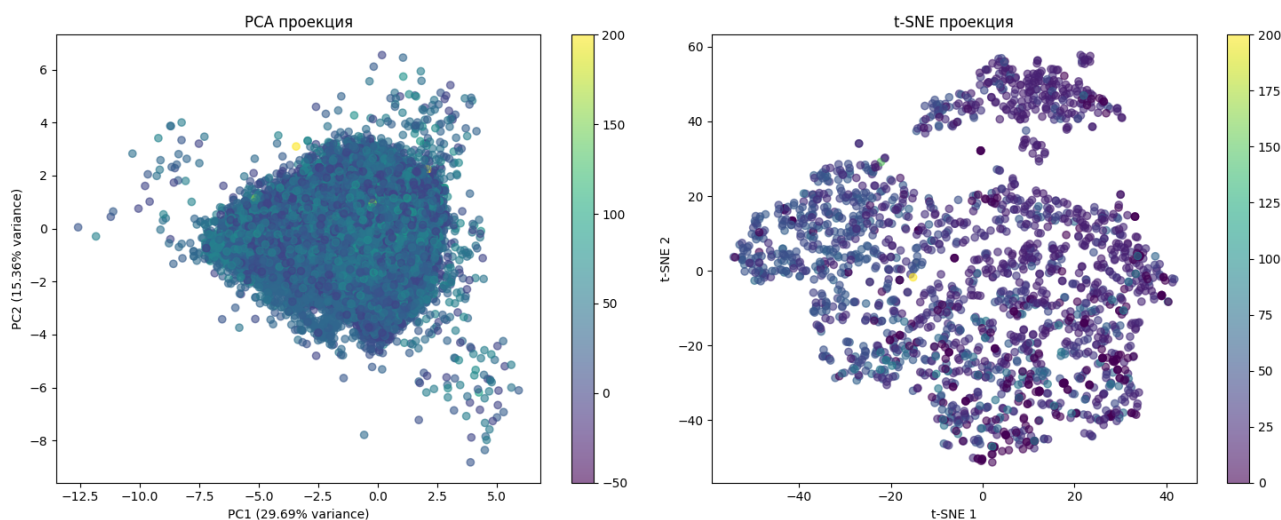


Рисунок 32 — PCA и t-SNE проекции

На PCA проекции почти полностью сливаются в один кластер, а t-SNE показывает более сложную структуру с несколькими группами.

Классы (уровни популярности) частично перекрываются, это указывает на сложность задачи классификации.

Музыкальные жанры формируют отдельные кластеры. Акустические характеристики создают естественные группы.

На обеих проекциях видны отдельные точки, удаленные от основных скоплений. Эти точки могут представлять уникальные музыкальные характеристики или выбросы.

Выводы:

1. Данные имеют сложную многомерную структуру.
2. Существуют естественные группы, соответствующие музыкальным жанрам.
3. Для задачи предсказания популярности важно учитывать как акустические, так и жанровые характеристики.

ВЫВОД

Проведенный анализ данных (EDA) выявил и устранил ключевые проблемы, подготовив набор для прогнозирования популярности музыкальных треков. Основные сложности включали пропуски в категориальных признаках (`artists`, `album_name`, `track_name`, `track_genre`), которые были удалены, а числовые пропуски заменены медианой для минимизации искажения статистических характеристик.

Выбросы в признаках `duration_ms` и акустических характеристиках обработаны адаптивно: для `duration_ms` установлены физические границы (1 секунда — 20 минут), для остальных — сочетание методов Winsorization и IQR-метода с учетом степени асимметрии распределений.

Классовый дисбаланс, где высокопопулярные треки составляли всего 0,9% выборки (соотношение 1:113), устранен генерацией 58 273 синтетических записей с добавлением шума 15–25%, что улучшило соотношение классов до 1:2.

Для обработки категориальных признаков применен Label Encoding для высококардинальных (`artists`, `album_name`, `track_name`) для снижения вычислительной сложности и One-Hot Encoding для `track_genre` с ограниченным числом уникальных значений. Масштабирование числовых признаков адаптировано под распределение: StandardScaler для нормальных (`danceability`, `energy`, `valence`, `tempo`), RobustScaler для сильно асимметричных (`instrumentalness`, `acousticness`) и MinMaxScaler для умеренно асимметричных (`speechiness`, `liveness`, `loudness`).

Анализ информативности показал высокую значимость акустических характеристик (`danceability`, `energy`, `loudness`, `valence`), жанра (`track_genre`) и длительности трека (`duration_ms`), тогда как идентификаторы (`track_id`, `index`) и именные признаки (`track_name`) оказались малоинформативными.

Признаки `artists`, `album_name` и `track_name` представляют риск утечки данных, так как могут косвенно отражать популярность через известность

исполнителей.

Межпризнаковые корреляции выявили сильную положительную связь между energy и loudness (коэффициент корреляции 0,72), умеренную между danceability и valence (0,48), а также отрицательную зависимость acousticness и energy (−0,65).

На основе структуры данных и выявленных особенностей приоритетными алгоритмами для прогнозирования являются Random Forest и XGBoost, которые устойчивы к шуму и способны обрабатывать нелинейные зависимости. Для компенсации дисбаланса классов рекомендовано использовать `class_weight='balanced'` и стратифицированную кросс-валидацию с контролем метрик Precision, Recall и F1-score.

Данные приведены в соответствие с требованиями машинного обучения и готовы к использованию в задачах прогнозирования популярности музыкальных треков.